شرح سطر بسطر ١ — الأساس: الإعداد والتطبيق والتنقل والثيم والنصوص

مشروع رِزمة · Flutter · كل سطر مع سبب كتابته بهذه الطريقة

يشرح هذا الملف «الأساس» الذي تقوم عليه كل الشاشات: إعداد المشروع (`pubspec.yaml`)، نقطة الدخول، جذر التطبيق، المسارات، الألوان، أنماط النص، ألوان شخصية الطالب، وملف النصوص العربية.

طريقة القراءة: كل مقطع كود يتبعه سؤالان: ماذا يفعل؟ ولماذا كُتب بهذه الطريقة؟ الأسطر الفارغة لا تُشرح.

تذكير بالاتجاه: التطبيق عربي (RTL)، فأول عنصر في أي `Row` يظهر على اليمين.

pubspec.yaml — الحزم والخط والأصول

pubspec.yaml

السطر 36

```
36    cupertino_icons: ^1.0.8
```

ماذا يفعل؟ حزمة أيقونات iOS الافتراضية من القالب.

لماذا بهذه الطريقة؟ تأتي مع كل مشروع Flutter؛ لا نستعملها حاليًا لكنها لا تضر.

السطر 37

```
37    flutter_svg: ^2.3.0
```

ماذا يفعل؟ `flutter_svg` لعرض ملفات SVG.

لماذا بهذه الطريقة؟ كل الأيقونات والشخصية مُصدَّرة من Figma بصيغة SVG؛ تبقى حادّة بأي حجم وأخفّ من الصور النقطية.

الأسطر 38–39

```
38    flutter_localizations:
39      sdk: flutter
```

ماذا يفعل؟ `flutter_localizations` من الـ SDK نفسه.

لماذا بهذه الطريقة؟ تعرّب مكوّنات Material (اتجاه RTL وأسماء الأزرار) وتتطلب كتابة `sdk: flutter`.

السطر 40

```
40    flutter_riverpod: ^2.6.1
```

ماذا يفعل؟ `flutter_riverpod` بإصدار 2.6.1.

لماذا بهذه الطريقة؟ إدارة الحالة. استعملنا 2.6.1 لا x.3 لأن Flutter 3.38 يثبّت `meta 1.17` وإصدارات Riverpod 3 تتعارض معه عبر حزمة `test`.

السطر 41

```
41    go_router: ^17.5.0
```

ماذا يفعل؟ `go_router` للتنقل.

لماذا بهذه الطريقة؟ يدعم المسارات المسمّاة وقواعد التحويل (redirect) ويصلح للويب وللجوال.

الأسطر 42–43

```
42    drift: '>=2.31.0 <2.32.0'
43    drift_flutter: '>=0.2.0 <0.3.0'
```

ماذا يفعل؟ `drift` و`drift_flutter` بنطاق إصدارات ضيّق.

لماذا بهذه الطريقة؟ قاعدة SQLite محلية. النطاق الضيّق (2.31 و0.2.x) لأن أحدث `drift_dev` يتطلب `analyzer` أحدث يتعارض مع `meta` المثبّتة في Flutter الحالي.

السطر 44

```
44    path_provider: ^2.1.6
```

ماذا يفعل؟ `path_provider`.

لماذا بهذه الطريقة؟ تحتاجه `drift_flutter` لمعرفة مجلد حفظ ملف القاعدة على الجهاز.

السطر 45

```
45    crypto: ^3.0.7
```

ماذا يفعل؟ `crypto` (SHA-256).

لماذا بهذه الطريقة؟ لتجزئة الرمز السري فلا يُخزَّن نصًّا أبدًا.

السطر 46

```
46    uuid: ^4.6.0
```

ماذا يفعل؟ `uuid`.

لماذا بهذه الطريقة؟ لتوليد معرّف فريد لكل ملف طالب.

السطر 47

```
47    share_plus: ^13.3.1
```

ماذا يفعل؟ `share_plus`.

لماذا بهذه الطريقة؟ لمشاركة رمز الطالب في المرحلة 6 (مضافة سلفًا حسب الخطة).

الأسطر 49–51

```
49  dev_dependencies:
50    flutter_test:
51      sdk: flutter
```

ماذا يفعل؟ قسم الاعتماديات التطويرية يبدأ بـ `flutter_test`.

لماذا بهذه الطريقة؟ لتشغيل اختبارات الوحدة والواجهة.

السطر 58

```
58    flutter_lints: ^6.0.0
```

ماذا يفعل؟ `flutter_lints`.

لماذا بهذه الطريقة؟ مجموعة قواعد تنظيف الكود الموصى بها.

الأسطر 59–60

```
59    drift_dev: '>=2.31.0 <2.32.0'
60    build_runner: ^2.4.13
```

ماذا يفعل؟ `drift_dev` و`build_runner`.

لماذا بهذه الطريقة؟ يولّدان ملفات `*.g.dart` (فئات الجداول والاستعلامات). النطاق مطابق لـ `drift` ليعملا معًا.

السطر 71

```
71    uses-material-design: true
```

ماذا يفعل؟ تفعيل خط أيقونات Material.

لماذا بهذه الطريقة؟ سطر القالب؛ بعض المكوّنات (مثل `Icons.lock`) تحتاجه.

الأسطر 73–76

```
73    assets:
74      - assets/images/brand/
75      - assets/icons/
76      - assets/content/
```

ماذا يفعل؟ تسجيل مجلدات الأصول: العلامة التجارية، الأيقونات، والمحتوى (الكتالوج).

لماذا بهذه الطريقة؟ Flutter لا يرى أي ملف غير مسجَّل. المجلد بشرطة مائلة في آخره يضيف كل ملفاته دفعة واحدة.

الأسطر 78–80

```
78    fonts:
79      - family: Cairo
80        fonts:
```

ماذا يفعل؟ تعريف عائلة الخط `Cairo`.

لماذا بهذه الطريقة؟ الخط داخل التطبيق فيعمل دون إنترنت من أول تشغيل (بدل `google_fonts`).

الأسطر 81–88

```
        - asset: assets/fonts/Cairo-Regular.ttf
          weight: 400
        - asset: assets/fonts/Cairo-SemiBold.ttf
          weight: 600
        - asset: assets/fonts/Cairo-Bold.ttf
          weight: 700
        - asset: assets/fonts/Cairo-ExtraBold.ttf
          weight: 800
```

ماذا يفعل؟ أربعة ملفات خط بأوزان 400 و600 و700 و800.

لماذا بهذه الطريقة؟ تطابق أوزان Figma (Regular وSemiBold وBold وExtraBold) فيختار Flutter الملف الصحيح عند `FontWeight.w800` مثلًا.

lib/main.dart — نقطة الدخول

`lib/main.dart`

الأسطر 1–2

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
```

ماذا يفعل؟ استيراد Material وRiverpod.

لماذا بهذه الطريقة؟ `runApp` من Material، و`ProviderScope` من Riverpod.

السطر 3

```
import 'app/rizma_app.dart';
```

ماذا يفعل؟ استيراد جذر التطبيق.

لماذا بهذه الطريقة؟ نفصل «التطبيق» عن «التشغيل» ليبقى `main` قصيرًا.

الأسطر 5–7

```
void main() {
  runApp(const ProviderScope(child: RizmaApp()));
}
```

ماذا يفعل؟ الدالة `main` تشغّل التطبيق داخل `ProviderScope`.

لماذا بهذه الطريقة؟ `ProviderScope` حاوية كل المزوّدات (providers)؛ بدونها لا يعمل Riverpod. `const` لأن الودجت لا يتغيّر.

lib/app/rizma_app.dart — جذر التطبيق

`lib/app/rizma_app.dart`

الأسطر 1–3

```
import 'package:flutter/material.dart';
import 'package:flutter_localizations/flutter_localizations.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
```

ماذا يفعل؟ استيراد Material وحزمة التعريب وRiverpod.

لماذا بهذه الطريقة؟ حزمة التعريب تعطي «المندوبين» (delegates) الذين يترجمون مكوّنات Flutter.

الأسطر 5–8

```
import '../core/strings/ar.dart';
import '../core/theme/app_colors.dart';
import '../core/theme/app_theme.dart';
import 'router.dart';
```

ماذا يفعل؟ استيراد النصوص والألوان والثيم والراوتر.

لماذا بهذه الطريقة؟ الاسم من `ArStrings` (لا نص ثابت)، والثيم والراوتر مزوّدان من ملفاتهما.

الأسطر 10–11

```
/// أقصى عرض للمحتوى (هاتف كبير).
const _maxContentWidth = 480.0;
```

ماذا يفعل؟ ثابت خاص `_maxContentWidth = 480` مع تعليقه.

لماذا بهذه الطريقة؟ رقم واحد مسمّى بدل 480 مبعثرة. الصنف الخاص (بشرطة سفلية) يبقى داخل الملف.

الأسطر 13–14

```
class RizmaApp extends ConsumerWidget {
  const RizmaApp({super.key});
```

ماذا يفعل؟ ودجت `RizmaApp` من نوع `ConsumerWidget`.

لماذا بهذه الطريقة؟ `ConsumerWidget` يعطينا `ref` لقراءة المزوّدات. نحتاجه لقراءة الراوتر.

الأسطر 16–17

```
  @override
  Widget build(BuildContext context, WidgetRef ref) {
```

ماذا يفعل؟ دالة `build` بمعاملين: السياق و`ref`.

لماذا بهذه الطريقة؟ توقيع `ConsumerWidget`.

السطر 18

```
    return MaterialApp.router(
```

ماذا يفعل؟ `MaterialApp.router` بدل `MaterialApp` العادي.

لماذا بهذه الطريقة؟ لأن التنقل عبر go_router (يعطي `routerConfig`).

الأسطر 19–20

```
19        title: ArStrings.appName,
20        debugShowCheckedModeBanner: false,
```

ماذا يفعل؟ عنوان التطبيق من `ArStrings` وإخفاء شريط DEBUG.

لماذا بهذه الطريقة؟ العنوان يظهر في مبدّل مهام النظام.

السطر 21

```
21        theme: AppTheme.light,
```

ماذا يفعل؟ الثيم `AppTheme.light`.

لماذا بهذه الطريقة؟ كل الألوان والخط من مكان واحد.

الأسطر 22–23

```
22        locale: const Locale('ar'),
23        supportedLocales: const [Locale('ar')],
```

ماذا يفعل؟ اللغة `ar` وحصر اللغات المدعومة فيها.

لماذا بهذه الطريقة؟ هذا هو الذي يقلب التطبيق كله إلى RTL تلقائيًا فلا نحتاج `Directionality` يدويًا.

الأسطر 24–28

```
24        localizationsDelegates: const [
25          GlobalMaterialLocalizations.delegate,
26          GlobalWidgetsLocalizations.delegate,
27          GlobalCupertinoLocalizations.delegate,
28        ],
```

ماذا يفعل؟ ثلاثة مندوبين للتعريب: Material وWidgets وCupertino.

لماذا بهذه الطريقة؟ Widgets يحدّد اتجاه النص، وMaterial يترجم الحقول والأزرار، وCupertino لمكوّنات iOS. حذف أحدها يسبب أخطاء أو اتجاهًا خاطئًا.

الأسطر 29–40

```
        // تطبيق هاتف: على شاشة عريضة (كروم/حاسوب لوحي) نحصر العرض في عمود
        // بعرض هاتف بدل أن تتمدّد البطاقات والشريط السفلي على كامل العرض.
        builder: (context, child) => ColoredBox(
          color: AppColors.mute,
          child: Align(
            alignment: Alignment.topCenter,
            child: ConstrainedBox(
              constraints: const BoxConstraints(maxWidth: _maxContentWidth),
              child: child,
            ),
          ),
        ),
```

ماذا يفعل؟ `builder`: نلفّ كل الشاشات في `ColoredBox` بلون `mute` ثم `Align` أعلى الوسط ثم `ConstrainedBox` بأقصى عرض 480.

لماذا بهذه الطريقة؟ تطبيق هاتف: على كروم أو حاسوب لوحي كانت البطاقات والشريط السفلي تتمدّد على كامل العرض (1200 بكسل مثلًا). الآن يظهر عمود بعرض هاتف في الوسط وجانباه رماديان. لأن `builder` فوق `Navigator` فالأوراق السفلية والرسائل المنبثقة تتقيّد بالعرض نفسه. على الهاتف الحقيقي (أضيق من 480) لا أثر له.

السطر 41

```
        routerConfig: ref.watch(routerProvider),
```

ماذا يفعل؟ ربط `routerConfig` بالراوتر المقروء عبر `ref.watch`.

لماذا بهذه الطريقة؟ `watch` وليس `read` ليُعاد البناء إن تغيّر الراوتر.

الأسطر 42–44

```
      );
    }
  }
```

ماذا يفعل؟ إغلاق `MaterialApp` والدالة والصنف.

لماذا بهذه الطريقة؟ نهاية الملف.

lib/app/routes.dart — أسماء المسارات

lib/app/routes.dart

الأسطر 1–2

```
/// مسارات التنقل، منفصلة عن `router.dart` نفسه حتى لا تعتمد الشاشات على الراوتر.
abstract final class AppRoutes {
```

ماذا يفعل؟ تعليق وصنف ثوابت `AppRoutes`.

لماذا بهذه الطريقة؟ ملف مستقل عن الراوتر كي تستورده الشاشات دون أن تستورد الراوتر (الذي يستورد الشاشات) فنتجنب اعتمادًا دائريًا. `abstract final` يمنع إنشاء كائن منه.

الأسطر 3–4

```
3     /// شاشة إقلاع صامتة تقرر إلى أين نذهب عند فتح التطبيق.
4     static const launch = '/';
```

ماذا يفعل؟ المسار `/` = شاشة الإقلاع.

لماذا بهذه الطريقة؟ لا تعرض شيئًا؛ تقرر إلى أين نذهب (ترحيب أم رئيسية أم اختيار ملف).

السطر 5

```
5     static const welcome = '/welcome';
```

ماذا يفعل؟ `/welcome` للترحيب.

لماذا بهذه الطريقة؟ لم يعد الترحيب في `/` لأن `/` صارت للإقلاع.

الأسطر 6–7

```
6     static const createStep1 = '/create/step1';
7     static const createStep2 = '/create/step2';
```

ماذا يفعل؟ خطوتا إنشاء الملف.

لماذا بهذه الطريقة؟ مسار لكل خطوة ليُعاد التوجيه بدقة إن فُتحت الخطوة 2 بلا بيانات الخطوة 1.

السطر 8

```
8     static const profiles = '/profiles';
```

ماذا يفعل؟ `/profiles` لاختيار الملف.

لماذا بهذه الطريقة؟ شاشة «من يتعلّم الآن؟».

السطر 9

```
9     static const pinPattern = '/pin/:id';
```

ماذا يفعل؟ نمط المسار `/pin/:id` .

لماذا بهذه الطريقة؟ الجزء `:id` متغيّر: معرّف الملف المراد فتحه.

السطر 10

```
10    static const home = '/home';
```

ماذا يفعل؟ `/home` للرئيسية.

لماذا بهذه الطريقة؟ وجهة الدخول النهائية.

الأسطر 12–11

```
11    static const account = '/account';
12    static const studentCode = '/account/code';
```

ماذا يفعل؟ `/account` لحسابي و `/account/code` لرمز الطالب.

لماذا بهذه الطريقة؟ مسار الرمز تحت مسار الحساب لأنه شاشة فرعية منه؛ يُفتح بـ `push` فيعود زر الرجوع إلى حسابي.

السطر 14

```
14    static String pin(String profileId) => '/pin/$profileId';
```

ماذا يفعل؟ دالة `pin(profileId)` تبني مسار شاشة الرمز لملف معيّن.

لماذا بهذه الطريقة؟ تجنّب لصق النص `"/pin/$id"` في الشاشات (خطأ مطبعي واحد يكسر التنقل).

السطر 15

```
15  }
```

ماذا يفعل؟ إغلاق الصنف.

لماذا بهذه الطريقة؟ نهاية الملف.

lib/app/router.dart — الراوتر وقواعد التحويل

lib/app/router.dart

الأسطر 3–1

```
1   import 'package:flutter/widgets.dart';
2   import 'package:flutter_riverpod/flutter_riverpod.dart';
3   import 'package:go_router/go_router.dart';
```

ماذا يفعل؟ استيراد `widgets` (لنوع `BuildContext`) وRiverpod وgo_router.

لماذا بهذه الطريقة؟ نبني الراوتر داخل مزوّد. `widgets` وحدها تكفي ولا نحتاج Material كاملة.

الأسطر 16–5

```
5   import '../features/account/account_screen.dart';
6   import '../features/account/student_code_screen.dart';
7   import '../features/home/home_screen.dart';
8   import '../features/launch/launch_screen.dart';
9   import '../features/onboarding/create_profile_draft.dart';
10  import '../features/onboarding/create_profile_step1_screen.dart';
11  import '../features/onboarding/create_profile_step2_screen.dart';
12  import '../features/onboarding/welcome_screen.dart';
13  import '../features/profiles/active_profile_provider.dart';
14  import '../features/profiles/pin_entry_screen.dart';
15  import '../features/profiles/profile_picker_screen.dart';
16  import 'routes.dart';
```

ماذا يفعل؟ استيراد كل الشاشات والمسارات ومزوّد المسودة والملف النشط.

لماذا بهذه الطريقة؟ الراوتر هو المكان الوحيد الذي يعرف كل الشاشات (ومنها الآن حسابي ورمز الطالب).

السطر 18

```
18  final routerProvider = Provider<GoRouter>((ref) {
```

ماذا يفعل؟ `routerProvider` يبني `GoRouter` داخل `Provider`.

لماذا بهذه الطريقة؟ نضعه في مزوّد ليصل إلى `ref` فيقرأ المسودة والملف النشط في قواعد التحويل.

الأسطر 19–23

```
19    String? requireActiveProfile(BuildContext context, GoRouterState state) {
20      final active = ref.read(activeProfileIdProvider);
21      return active.hasValue && active.value == null ? AppRoutes.launch : null;
22    }
```

ماذا يفعل؟ دالة محلية `requireActiveProfile`: إن كانت قراءة الملف النشط جاهزة وقيمتها `null` تعيد مسار الإقلاع، وإلا `null` (لا تحويل).

لماذا بهذه الطريقة؟ كتبناها مرة واحدة وأعدنا استعمالها في ثلاثة مسارات (الرئيسية وحسابي ورمز الطالب) بدل تكرار المنطق.
`hasValue` تتأكد أن القراءة اكتملت فلا نحوّل خطأً أثناء التحميل.

الأسطر 24–26

```
24    return GoRouter(
25      initialLocation: AppRoutes.launch,
26      routes: [
```

ماذا يفعل؟ بداية `GoRouter`: المسار الابتدائي هو الإقلاع وبداية قائمة المسارات.

لماذا بهذه الطريقة؟ كل فتح للتطبيق يمر بالإقلاع فيقرر الوجهة.

الأسطر 27–31

```
27        // تقرر الوجهة عند فتح التطبيق (انظر launch_plan.dart).
28        GoRoute(
29          path: AppRoutes.launch,
30          builder: (context, state) => const LaunchScreen(),
31        ),
```

ماذا يفعل؟ مسار الإقلاع `/` يبني `LaunchScreen`.

لماذا بهذه الطريقة؟ التعليق يشير إلى `launch_plan.dart` الذي فيه القرار.

الأسطر 32–35

```
32        GoRoute(
33          path: AppRoutes.welcome,
34          builder: (context, state) => const WelcomeScreen(),
35        ),
```

ماذا يفعل؟ مسار الترحيب.

لماذا بهذه الطريقة؟ يبني `WelcomeScreen`.

الأسطر 36–39

```
      GoRoute(
        path: AppRoutes.createStep1,
        builder: (context, state) => const CreateProfileStep1Screen(),
      ),
```

ماذا يفعل؟ مسار الخطوة 1 (الاسم والصف).

لماذا بهذه الطريقة؟ يبني `CreateProfileStep1Screen`.

الأسطر 40–42

```
      GoRoute(
        path: AppRoutes.createStep2,
        // بلا بيانات الخطوة 1 لا معنى للخطوة 2 (مثلًا بعد إعادة فتح التطبيق).
```

ماذا يفعل؟ مسار الخطوة 2 وتعليق سببه.

لماذا بهذه الطريقة؟ الخطوة 2 تعتمد على بيانات الخطوة 1.

الأسطر 43–46

```
        redirect: (context, state) =>
            ref.read(createProfileDraftProvider) == null
            ? AppRoutes.createStep1
            : null,
```

ماذا يفعل؟ `redirect`: إن لم توجد مسودة من الخطوة 1 نذهب للخطوة 1، وإلا `null` (لا تحويل).

لماذا بهذه الطريقة؟ يمنع فتح الخطوة 2 بلا بيانات (مثلًا بعد إعادة تحميل الصفحة على الويب). `null` معناه «ابقَ حيث أنت» في go_router.

الأسطر 47–48

```
        builder: (context, state) => const CreateProfileStep2Screen(),
      ),
```

ماذا يفعل؟ بناء `CreateProfileStep2Screen` وإغلاق المسار.

لماذا بهذه الطريقة؟ الشاشة نفسها.

الأسطر 49–52

```
      GoRoute(
        path: AppRoutes.profiles,
        builder: (context, state) => const ProfilePickerScreen(),
      ),
```

ماذا يفعل؟ مسار `/profiles`.

لماذا بهذه الطريقة؟ يبني شاشة اختيار الملف.

الأسطر 53–57

```
      GoRoute(
        path: AppRoutes.pinPattern,
        builder: (context, state) =>
            PinEntryScreen(profileId: state.pathParameters['id']!),
      ),
```

ماذا يفعل؟ مسار `/pin/:id`: يستخرج `id` من `pathParameters` ويمرّره لـ `PinEntryScreen`.

لماذا بهذه الطريقة؟ علامة `!` لأن المسار لا يتطابق إلا بوجود `id`.

الأسطر 58–59

```
      // شاشات تحتاج ملفًا نشطًا (مثلًا فتح الرابط مباشرة بلا ملف): نعود
      // لشاشة الإقلاع لتقرر الوجهة.
```

ماذا يفعل؟ تعليق: الشاشات التي تحتاج ملفًا نشطًا.

لماذا بهذه الطريقة؟ بلا ملف نشط (مثلًا فتح الرابط مباشرة) لا معنى لها فنعود للإقلاع.

الأسطر 60–64

```
      GoRoute(
        path: AppRoutes.home,
        redirect: requireActiveProfile,
        builder: (context, state) => const HomeScreen(),
      ),
```

ماذا يفعل؟ مسار الرئيسية: `redirect: requireActiveProfile`.

لماذا بهذه الطريقة؟ نمرّر الدالة نفسها لا نتيجتها؛ go_router يستدعيها عند كل انتقال.

الأسطر 65–69

```
      GoRoute(
        path: AppRoutes.account,
        redirect: requireActiveProfile,
        builder: (context, state) => const AccountScreen(),
      ),
```

ماذا يفعل؟ مسار حسابي: نفس القاعدة ويبني `AccountScreen`.

لماذا بهذه الطريقة؟ لا حساب بلا ملف نشط.

الأسطر 70–74

```
      GoRoute(
        path: AppRoutes.studentCode,
        redirect: requireActiveProfile,
        builder: (context, state) => const StudentCodeScreen(),
      ),
```

ماذا يفعل؟ مسار رمز الطالب: نفس القاعدة ويبني `StudentCodeScreen`.

لماذا بهذه الطريقة؟ الرمز يخص الملف النشط.

الأسطر 75–77

```
    ],
  );
});
```

ماذا يفعل؟ إغلاق قائمة المسارات و`GoRouter` والمزوّد.

لماذا بهذه الطريقة؟ نهاية الملف.

lib/core/constants/app_assets.dart — مسارات الأصول

lib/core/constants/app_assets.dart

الأسطر 1–2

```
/// مسارات الأصول (SVG) المصدَّرة من Figma — مكان واحد لكل المسارات.
abstract final class AppAssets {
```

ماذا يفعل؟ تعليق وصنف `AppAssets`.

لماذا بهذه الطريقة؟ مكان واحد لكل مسارات الملفات فلا نكتب النصوص في الشاشات.

الأسطر 3–4

```
  static const _brand = 'assets/images/brand';
  static const _icons = 'assets/icons';
```

ماذا يفعل؟ ثابتان خاصان لمجلدي الأصول.

لماذا بهذه الطريقة؟ تغيير اسم مجلد = تعديل سطر واحد.

الأسطر 6–11

```
  // الهوية
  static const tatreezBand = '$_brand/tatreez_band.svg';
  static const mascot150 = '$_brand/mascot_150.svg';
  static const mascot56 = '$_brand/mascot_56.svg';
  static const mascot40 = '$_brand/mascot_40.svg';
  static const logoOnDark = '$_brand/logo_on_dark.svg';
```

ماذا يفعل؟ أصول الهوية: شريط التطريز، الشخصية بثلاثة أحجام، والشعار على الداكن.

لماذا بهذه الطريقة؟ الشخصية مُصدَّرة من Figma بثلاثة ملفات؛ نستعمل منها `mascot150` أساسًا (ويُلوَّن وقت العرض).

الأسطر 13–17

```
  // شاشة الترحيب
  static const bookTile = '$_icons/book_tile.svg';
  static const cloudOff = '$_icons/cloud_off.svg';
  static const target = '$_icons/target.svg';
  static const fwdOnboarding = '$_icons/fwd_onboarding.svg';
```

ماذا يفعل؟ أيقونات شاشة الترحيب.

لماذا بهذه الطريقة؟ مجمّعة بحسب الشاشة.

الأسطر 19–21

```
19    // شاشة اختيار الصف
20    static const back = '$_icons/back.svg';
21    static const fwdGrade = '$_icons/fwd_grade.svg';
```

ماذا يفعل؟ أيقونتا الرجوع والمتابعة في إنشاء الملف.

لماذا بهذه الطريقة؟ أيقونة الرجوع مرسومة بسهم لليمين = اتجاه الرجوع في RTL، فلا نقلبها برمجيًا.

الأسطر 23–27

```
23    // الشريط السفلي
24    static const navUser = '$_icons/nav_user.svg';
25    static const navChart = '$_icons/nav_chart.svg';
26    static const navBook = '$_icons/nav_book.svg';
27    static const navHome = '$_icons/nav_home.svg';
```

ماذا يفعل؟ أيقونات الشريط السفلي.

لماذا بهذه الطريقة؟ مرسومة بخط ولون واحد فتُصبغ بـ `ColorFilter`.

الأسطر 29–41

```
29    // الشاشة الرئيسية
30    static const starFilled = '$_icons/star_filled.svg';
31    static const flame = '$_icons/flame.svg';
32    static const calcCard = '$_icons/calc_card.svg';
33    static const play = '$_icons/play.svg';
34    static const warn = '$_icons/warn.svg';
35    static const fwdRow = '$_icons/fwd_row.svg';
36    static const bolt = '$_icons/bolt.svg';
37    static const leaf = '$_icons/leaf.svg';
38    static const chat = '$_icons/chat.svg';
39    static const bookSubject = '$_icons/book_subject.svg';
40    static const calcSubject = '$_icons/calc_subject.svg';
41  }
```

ماذا يفعل؟ أيقونات الرئيسية ثم إغلاق الصنف.

لماذا بهذه الطريقة؟ أسماء وصفية (`calcCard` و`calcSubject`) لأن الأيقونة مُصدَّرة بحجمين.

lib/core/theme/app_colors.dart — ألوان التصميم

`lib/core/theme/app_colors.dart`

القيمة `0xFF` تعني «معتم بالكامل» ثم ستة أرقام سداسية عشرية للون. كل لون منسوخ من متغيّرات Figma.

السطر 1

```
1   import 'package:flutter/painting.dart';
```

ماذا يفعل؟ استيراد `painting` فقط.

لماذا بهذه الطريقة؟ الملف يحتاج `Color` فقط فنستورد الأخف.

الأسطر 3–4

```
3   /// ألوان التصميم كما هي في Figma (Color/*).
4   abstract final class AppColors {
```

ماذا يفعل؟ تعليق وصنف الثوابت.

لماذا بهذه الطريقة؟ `abstract final` : لا إنشاء كائن ولا وراثة. نغيّر اللون من مكان واحد.

الأسطر 5–12

```
5     // الخلفيات والأسطح
6     static const ground = Color(0xFFF4F5F1);
7     static const light = Color(0xFFF4F1E8);
8     static const card = Color(0xFFFFFFFF);
9     static const mute = Color(0xFFE6EAE5);
10    static const line = Color(0xFFE1E6E2);
11    static const disabled = Color(0xFFC9D2CD);
12    static const placeholder = Color(0xFF9AA7A1);
```

ماذا يفعل؟ الأسطح: خلفية الشاشات `ground` ، نص على الداكن `light` ، بطاقة `card` ، رمادي ثانوي `mute` ، حدود `line` ، معطّل `disabled` ، ونص إرشادي `placeholder` .

لماذا بهذه الطريقة؟ التسمية بالدور لا باللون، فتبقى صحيحة لو تغيّر اللون. `placeholder` أضفناه للنص الإرشادي في الحقول.

الأسطر 14–16

```
14    // النصوص
15    static const ink = Color(0xFF0F2A2E);
16    static const soft = Color(0xFF55655F);
```

ماذا يفعل؟ النصوص: `ink` الأساسي و `soft` الباهت.

لماذا بهذه الطريقة؟ درجتان تكفيان لتسلسل بصري واضح.

الأسطر 18–22

```
18    // الأخضر (اللون الأساسي)
19    static const teal = Color(0xFF177A63);
20    static const tealDark = Color(0xFF0E5C4A);
21    static const tealTint = Color(0xFFDCEFE6);
22    static const onTeal = Color(0xFFD5EEE3);
```

ماذا يفعل؟ عائلة الأخضر: `teal` الأساسي و `tealDark` و `tealTint` و `onTeal` .

لماذا بهذه الطريقة؟ `Tint` للخلفيات الناعمة، و `Dark` لمسار شريط التقدّم على الأخضر، و `onTeal` للنص فوق الأخضر.

الأسطر 24–28

```
24    // العنبري
25    static const amber = Color(0xFFE9A23B);
26    static const amberDark = Color(0xFF8F540A);
27    static const amberTint = Color(0xFFFCEFD6);
28    static const amberLine = Color(0xFFF1D29C);
```

ماذا يفعل؟ العنبري: `amber` و`amberDark` و`amberTint` و`amberLine`.

لماذا بهذه الطريقة؟ للنجوم وبطاقة المراجعة.

الأسطر 30–32

```
30    // الأزرق
31    static const blue = Color(0xFF2F66C9);
32    static const blueTint = Color(0xFFDEE7F8);
```

ماذا يفعل؟ الأزرق.

لماذا بهذه الطريقة؟ للعلوم.

الأسطر 34–36

```
34    // المرجاني
35    static const coral = Color(0xFFB94F2A);
36    static const coralTint = Color(0xFFF8E1D7);
```

ماذا يفعل؟ المرجاني.

لماذا بهذه الطريقة؟ للعربية وسلسلة الأيام.

الأسطر 38–41

```
38    // البنفسجي
39    static const purple = Color(0xFF6A4FB3);
40    static const purpleTint = Color(0xFFE9E3F6);
41  }
```

ماذا يفعل؟ البنفسجي ثم إغلاق الصنف.

لماذا بهذه الطريقة؟ للإنجليزية.

lib/core/theme/app_text_styles.dart — أنماط النص

`lib/core/theme/app_text_styles.dart`

اسم كل نمط يحمل حجمه (مثل `heading22`) ليطابق أسماء الأنماط في Figma.

الأسطر 1–3

```dart
import 'package:flutter/painting.dart';

import 'app_colors.dart';
```

ماذا يفعل؟ استيراد `painting` والألوان.

لماذا بهذه الطريقة؟ كل نمط يحمل لونه الافتراضي.

الأسطر 5–7

```dart
/// أنماط النص كما في Figma – كلها بخط Cairo.
/// الوزن: SemiBold=600، Bold=700، ExtraBold=800.
abstract final class AppTextStyles {
```

ماذا يفعل؟ تعليق وبداية الصنف.

لماذا بهذه الطريقة؟ الأوزان: SemiBold=600 وBold=700 وExtraBold=800.

السطر 8

```dart
  static const fontFamily = 'Cairo';
```

ماذا يفعل؟ ثابت اسم الخط `Cairo`.

لماذا بهذه الطريقة؟ يُستعمل في الثيم وفي أي `TextStyle` مكتوب يدويًا.

الأسطر 10–23

```dart
  static TextStyle _cairo(
    double size,
    FontWeight weight,
    double height,
    Color color,
  ) {
    return TextStyle(
      fontFamily: fontFamily,
      fontSize: size,
      fontWeight: weight,
      height: height,
      color: color,
    );
  }
```

ماذا يفعل؟ دالة خاصة `_cairo(size, weight, height, color)` تبني `TextStyle`.

لماذا بهذه الطريقة؟ تمنع تكرار الإعدادات عشرين مرة. `height` مضاعف لحجم الخط ويطابق `line-height` في Figma.

الأسطر 25–31

```
25    // الشعار والعناوين
26    static final wordmark40 = _cairo(40, FontWeight.w800, 1.3, AppColors.light);
27    static final tagline18 = _cairo(18, FontWeight.w700, 1.6, AppColors.light);
28    static final title26 = _cairo(26, FontWeight.w800, 1.35, AppColors.ink);
29    static final heading22 = _cairo(22, FontWeight.w800, 1.35, AppColors.ink);
30    static final cardTitle20 = _cairo(20, FontWeight.w800, 1.4, AppColors.card);
31    static final section18 = _cairo(18, FontWeight.w800, 1.4, AppColors.ink);
```

ماذا يفعل؟ أنماط الشعار والعناوين: `wordmark40` و`tagline18` و`title26` و`heading22` و`cardTitle20` و`section18`.

لماذا بهذه الطريقة؟ `static final` وليس `const` لأن الدالة `_cairo` ليست ثابتة وقت الترجمة.

الأسطر 33–37

```
33    // النص الأساسي
34    static final body16Strong = _cairo(16, FontWeight.w800, 1.5, AppColors.ink);
35    static final body15 = _cairo(15, FontWeight.w700, 1.5, AppColors.ink);
36    static final label14 = _cairo(14, FontWeight.w700, 1.45, AppColors.soft);
37    static final caption13 = _cairo(13, FontWeight.w600, 1.45, AppColors.soft);
```

ماذا يفعل؟ النص الأساسي: `body16Strong` و`body15` و`label14` و`caption13`.

لماذا بهذه الطريقة؟ أحجام 13–16 تغطي معظم الواجهة.

الأسطر 38–43

```
38    static final captionBold13 = _cairo(
39      13,
40      FontWeight.w700,
41      1.45,
42      AppColors.soft,
43    );
```

ماذا يفعل؟ `captionBold13` (13 بوزن 700).

لماذا بهذه الطريقة؟ منفصل عن `caption13` (وزن 600) لأن التصميم يستعمل الوزنين في أماكن مختلفة.

السطر 44

```
44    static final percent12 = _cairo(12, FontWeight.w800, 1.4, AppColors.ink);
```

ماذا يفعل؟ `percent12` لنسب التقدّم.

لماذا بهذه الطريقة؟ خط صغير سميك لرقم قصير.

الأسطر 46–51

```
46    // مكوّنات محدّدة
47    static final button16 = _cairo(16, FontWeight.w800, 1.5, AppColors.card);
48    static final input17 = _cairo(17, FontWeight.w700, 1.5, AppColors.ink);
49    static final gradeNumber24 = _cairo(24, FontWeight.w800, 1.3, AppColors.ink);
50    static final chip13 = _cairo(13, FontWeight.w700, 1.5, AppColors.ink);
51  }
```

ماذا يفعل؟ أنماط مكوّنات: `button16` و`input17` و`gradeNumber24` و`chip13` ثم إغلاق الصنف.

لماذا بهذه الطريقة؟ للتغيير اللوني نستعمل `copyWith(color:)` بدل نمط جديد لكل لون.

lib/core/theme/app_theme.dart — الثيم العام

`lib/core/theme/app_theme.dart`

الأسطر 1–4

```
1   import 'package:flutter/material.dart';
2
3   import 'app_colors.dart';
4   import 'app_text_styles.dart';
```

ماذا يفعل؟ استيراد Material والألوان والأنماط.

لماذا بهذه الطريقة؟ ما يلزم لبناء `ThemeData`.

الأسطر 6–7

```
6   abstract final class AppTheme {
7     static ThemeData get light {
```

ماذا يفعل؟ صنف `AppTheme` وخاصية `light`.

لماذا بهذه الطريقة؟ خاصية (getter) لسهولة إضافة `dark` لاحقًا بنفس الشكل.

الأسطر 8–17

```
8       final colorScheme =
9           ColorScheme.fromSeed(
10            seedColor: AppColors.teal,
11            brightness: Brightness.light,
12          ).copyWith(
13            primary: AppColors.teal,
14            onPrimary: AppColors.card,
15            surface: AppColors.ground,
16            onSurface: AppColors.ink,
17          );
```

ماذا يفعل؟ نولّد `ColorScheme` من لون البذرة `teal` ثم نثبّت `primary` و`onPrimary` و`surface` و`onSurface`.

لماذا بهذه الطريقة؟ `fromSeed` يعطي لوحة كاملة لمكوّنات Material التي لا نخصّصها، ثم نفرض ألواننا حتى لا يعدّل Material الأخضر.

الأسطر 19–20

```
19      return ThemeData(
20        useMaterial3: true,
```

ماذا يفعل؟ نبني `ThemeData` بـ Material 3.

لماذا بهذه الطريقة؟ الافتراضي الحديث.

السطر 21

```
21        colorScheme: colorScheme,
```

ماذا يفعل؟ تمرير `colorScheme`.

لماذا بهذه الطريقة؟ مصدر الألوان للمكوّنات الجاهزة.

السطر 22

```
22        scaffoldBackgroundColor: AppColors.ground,
```

ماذا يفعل؟ لون خلفية كل `Scaffold` هو `ground`.

لماذا بهذه الطريقة؟ حتى لا نكرر `backgroundColor` في كل شاشة.

السطر 23

```
23        fontFamily: AppTextStyles.fontFamily,
```

ماذا يفعل؟ `fontFamily: Cairo`.

لماذا بهذه الطريقة؟ أي `Text` بلا نمط يستعمل Cairo (الخط المحلي).

الأسطر 24–26

```
24      );
25    }
26  }
```

ماذا يفعل؟ إغلاق الأقواس.

لماذا بهذه الطريقة؟ نهاية الصنف.

lib/core/theme/profile_palette.dart — ألوان شخصية الطالب

lib/core/theme/profile_palette.dart

القيم من `design_spec/README.md`. الشخصية الأصلية خضراء، والبقية تُحاكى باستبدال لونين فيها وقت الرسم (انظر `mascot.dart`).

الأسطر 1–3

```dart
import 'package:flutter/painting.dart';

import '../db/enums.dart';
```

ماذا يفعل؟ استيراد `painting` وتعداد `ProfileColor`.

لماذا بهذه الطريقة؟ التعداد معرَّف في قاعدة البيانات لأن اللون يُخزَّن مع الملف.

الأسطر 5–7

```dart
/// ألوان شخصية الطالب الخمسة (من مواصفات التصميم).
/// يحلّان محل لوني الشخصية الأصلية (teal) عبر `ColorMapper`.
extension ProfileColorPalette on ProfileColor {
```

ماذا يفعل؟ `extension` على `ProfileColor`.

لماذا بهذه الطريقة؟ الامتداد يضيف خصائص للتعداد دون تعديله: نكتب `profile.color.body`.

الأسطر 8–15

```dart
  /// لون جسم الشخصية.
  Color get body => switch (this) {
    ProfileColor.teal => const Color(0xFF177A63),
    ProfileColor.coral => const Color(0xFFB94F2A),
    ProfileColor.blue => const Color(0xFF2F66C9),
    ProfileColor.purple => const Color(0xFF6A4FB3),
    ProfileColor.amber => const Color(0xFFB07316),
  };
```

ماذا يفعل؟ `body`: لون جسم الشخصية لكل لون.

لماذا بهذه الطريقة؟ `switch` تعبيرية شاملة: لو أضفنا لونًا جديدًا يرفض المترجم الكود حتى نعرّف قيمته.

الأسطر 17–24

```dart
  /// لون الزاوية المطوية.
  Color get fold => switch (this) {
    ProfileColor.teal => const Color(0xFF7ED3B2),
    ProfileColor.coral => const Color(0xFFF3B9A3),
    ProfileColor.blue => const Color(0xFFA9C2F0),
    ProfileColor.purple => const Color(0xFFC9BCEB),
    ProfileColor.amber => const Color(0xFFF1D29C),
  };
```

ماذا يفعل؟ `fold`: لون الزاوية المطوية.

لماذا بهذه الطريقة؟ لونان فقط يتغيّران في الشخصية (الجسم والزاوية).

الأسطر 26–34

```
26    /// خلفية الأفاتار خلف الشخصية.
27    Color get avatarBackground => switch (this) {
28      ProfileColor.teal => const Color(0xFFDCEFE6),
29      ProfileColor.coral => const Color(0xFFF8E1D7),
30      ProfileColor.blue => const Color(0xFFDEE7F8),
31      ProfileColor.purple => const Color(0xFFE9E3F6),
32      ProfileColor.amber => const Color(0xFFFCEFD6),
33    };
34  }
```

ماذا يفعل؟ `avatarBackground` : الخلفية الفاتحة خلف الشخصية ثم إغلاق الامتداد.

لماذا بهذه الطريقة؟ تُستعمل في الأفاتار والمعاينة وبطاقات المواد.

lib/core/strings/ar.dart — كل النصوص العربية

lib/core/strings/ar.dart

الغرض: لا نص ظاهر داخل الشاشات؛ كلها هنا تمهيدًا للترجمة. النصوص الثابتة `const` ، والمركّبة دوال.

السطر 1

```
1   import '../db/enums.dart';
```

ماذا يفعل؟ استيراد التعداد `ProfileColor` .

لماذا بهذه الطريقة؟ لدالة `colorName` .

الأسطر 3–5

```
3   /// كل النصوص الظاهرة للمستخدم في مكان واحد، تمهيدًا للترجمة لاحقًا.
4   abstract final class ArStrings {
5     static const appName = 'رِزمة';
```

ماذا يفعل؟ تعليق وصنف `ArStrings` واسم التطبيق.

لماذا بهذه الطريقة؟ اسم التطبيق بالتشكيل «رِزمة» كما في الهوية.

الأسطر 7–9

```
7     // عام
8     static const back = 'رجوع';
9     static const continueLabel = 'متابعة';
```

ماذا يفعل؟ نصوص عامة: «رجوع» و«متابعة».

لماذا بهذه الطريقة؟ تُستعمل في أكثر من شاشة.

الأسطر 11–21

```
  // الترحيب
  static const welcomeTagline =
      'من المحتوى الذي يُقرأ… إلى التعلم الذي يُمارس.';
  static const welcomeValue1Title = 'رزمتك… بطريقة أوضح';
  static const welcomeValue1Subtitle = 'شرح، ملخص، فيديو وتدريب لكل درس.';
  static const welcomeValue2Title = 'تعلّم دون إنترنت';
  static const welcomeValue2Subtitle = 'نزّل الدرس مرة وتعلّم في أي وقت.';
  static const welcomeValue3Title = 'اعرف أين تحتاج مراجعة';
  static const welcomeValue3Subtitle = 'خطة قصيرة مبنية على نتيجتك.';
  static const welcomeStart = 'ابدأ';
  static const welcomeHaveProfile = 'لديّ ملف طالب على هذا الجهاز';
```

ماذا يفعل؟ نصوص شاشة الترحيب: الجملة التعريفية ونقاط القيمة الثلاث وزر «ابدأ» ورابط «لديّ ملف طالب».

لماذا بهذه الطريقة؟ منسوخة من Figma حرفيًا. الجمل الطويلة كُسرت على سطرين لقاعدة طول السطر فقط.

الأسطر 23–34

```
  // اختيار الصف (الخطوة 1 من 2)
  static const gradeStep = 'خطوة 1 من 2';
  static const gradeTitle = 'اختر صفّك';
  static const nameLabel = 'ما اسمك؟';
  static const nameHint = 'اكتب اسمك';
  static const gradeGroup14 = 'الصفوف 1–4';
  static const gradeGroup14Hint = 'صور وصوت أكثر';
  static const gradeGroup58 = 'الصفوف 5–8';
  static const gradeGroup58Hint = 'تدريبات وتقدّم';
  static const gradeGroup912 = 'الصفوف 9–12';
  static const gradeGroup912Hint = 'ملخّصات وامتحانات';
  static const gradeChangeLater = 'يمكنك تغيير الصف لاحقًا من حسابك.';
```

ماذا يفعل؟ نصوص الخطوة 1: العناوين وتسمية الحقل والنص الإرشادي ومجموعات الصفوف الثلاث وتلميح تغيير الصف لاحقًا.

لماذا بهذه الطريقة؟ «اكتب اسمك» أضفناه (غير موجود في Figma) لأن الحقل يبدأ فارغًا.

الأسطر 36–37

```
  static String continueWithGrade(int grade) => 'متابعة · الصف $grade';
  static String gradeSemantics(int grade) => 'الصف $grade';
```

ماذا يفعل؟ دالتان: نص زر المتابعة بعد اختيار الصف، ووصف الصف لقارئ الشاشة.

لماذا بهذه الطريقة؟ دوال لأنهما تحتويان رقمًا متغيّرًا.

الأسطر 39–54

```
39    // إعداد الملف (الخطوة 2 من 2)
40    static const setupStep = 'خطوة 2 من 2';
41    static const setupTitle = 'اجعل الملف لك';
42    static const setupPickColor = 'اختر لون شخصيتك';
43    static const setupPinTitle = 'رمز سري';
44    static const setupPinSubtitle = 'يمنع الآخرين من فتح ملفك على هذا الجهاز';
45    static const setupPinSwitch = 'تفعيل الرمز السري';
46    static const setupPinHint = '4–7 أرقام · يُنصح به للصفوف اختياري';
47    static const setupClassTitle = 'رمز الصف (اختياري)';
48    static const setupClassSubtitle = 'يعطيك إياه معلمك لتنضم إلى مجموعته';
49    static const setupClassHint = 'مثال: GZ-5A';
50    static const setupClassInvalid =
51        'رمز الصف غير صحيح. استخدم حروفًا إنجليزية وأرقامًا وشرطة، مثل GZ-5A.';
52    static const setupStart = 'ابدأ التعلّم';
53    static const setupChangeLater = 'يمكنك تغيير كل هذا لاحقًا من حسابك.';
54    static const setupCreateFailed = 'تعذّر إنشاء الملف، حاول مرة أخرى.';
```

ماذا يفعل؟ نصوص الخطوة 2: العنوان واللون والرمز السري ورمز الصف وزر «ابدأ التعلّم» ورسالة فشل الإنشاء.

لماذا بهذه الطريقة؟ `setupPinSwitch` اسم المفتاح لقارئ الشاشة وهو مختلف عن عنوان البطاقة «رمز سري» حتى لا يلتبس.

الأسطر 56–57

```
56    static String setupPreview(String name, int grade) =>
57        '$name · ${gradeName(grade)}';
```

ماذا يفعل؟ `setupPreview`: «الاسم · الصف».

لماذا بهذه الطريقة؟ تعرض معاينة الملف.

الأسطر 59–65

```
59    static String colorName(ProfileColor color) => switch (color) {
60      ProfileColor.teal => 'أخضر',
61      ProfileColor.coral => 'مرجاني',
62      ProfileColor.blue => 'أزرق',
63      ProfileColor.purple => 'بنفسجي',
64      ProfileColor.amber => 'عنبري',
65    };
```

ماذا يفعل؟ `colorName`: اسم كل لون بالعربية.

لماذا بهذه الطريقة؟ لقارئ الشاشة عند اختيار اللون. `switch` شاملة.

الأسطر 67–71

```
67    // لوحة الرمز السري
68    static const pinSetupTitle = 'اختر رمزًا سريًا من 4 أرقام';
69    static const pinConfirmTitle = 'أعد إدخال الرمز للتأكيد';
70    static const pinMismatch = 'الرمزان غير متطابقين، حاول مرة أخرى.';
71    static const pinDelete = 'حذف';
```

ماذا يفعل؟ نصوص ورقة الرمز السري: العنوانان ورسالة عدم التطابق وكلمة «حذف».

لماذا بهذه الطريقة؟ تُستعمل في إنشاء الرمز.

الأسطر 73–74

```
73    static String pinDotsSemantics(int filled, int length) =>
74        'أُدخل $filled من $length أرقام';
```

ماذا يفعل؟ `pinDotsSemantics`: «أُدخل N من 4 أرقام».

لماذا بهذه الطريقة؟ يقرأ قارئ الشاشة عدد الأرقام دون كشفها.

الأسطر 76–87

```
76    // (25) شاشة «من يتعلّم الآن؟»
77    static const pickerTitle = 'من يتعلّم الآن؟';
78    static const pickerSubtitle = 'اختر ملفك لتكمل من حيث توقفت.';
79    static const pickerNewTitle = 'طالب جديد';
80    static const pickerNewSubtitle = 'أضف ملفًا لأخ أو أخت';
81    static const pickerRestoreTitle = 'لديك رمز طالب؟';
82    static const pickerRestoreSubtitle = 'استعد تقدّمك على هذا الجهاز';
83    static const pickerPrivacy =
84        'الملفات محفوظة على هذا الجهاز · لا نطلب بريدًا ولا رقم هاتف';
85    static const restoreNeedsInternet =
86        'الاستعادة تحتاج اتصالًا بالإنترنت وستتوفّر قريبًا.';
87    static const profileProtected = 'محمي برمز سري';
```

ماذا يفعل؟ نصوص شاشة «من يتعلّم الآن؟»: العنوان والفرعي وبطاقة «طالب جديد» ورابط الاستعادة وملاحظة الخصوصية ورسالة الاستعادة وكلمة «محمي برمز سري».

لماذا بهذه الطريقة؟ رسالة الاستعادة صريحة أنها تحتاج إنترنتًا ولا نزيّف نجاحًا.

الأسطر 89–90

```
89    static String profileSemantics(String name, String grade, bool locked) =>
90        locked ? '$name، $grade، $profileProtected' : '$name، $grade';
```

ماذا يفعل؟ `profileSemantics`: وصف البطاقة لقارئ الشاشة.

لماذا بهذه الطريقة؟ إن كان الملف محميًا تُضاف العبارة.

الأسطر 92–99

```
92    /// «نجمة واحدة» / «نجمتان» / «3 نجوم» … «10 نجوم» / «11 نجمة».
93    static String stars(int n) {
94      if (n == 0) return '0 نجوم';
95      if (n == 1) return 'نجمة واحدة';
96      if (n == 2) return 'نجمتان';
97      if (n >= 3 && n <= 10) return '$n نجوم';
98      return '$n نجمة';
99    }
```

ماذا يفعل؟ `stars(n)`: صيغة الجمع العربية للنجوم.

لماذا بهذه الطريقة؟ 0 → «0 نجوم»، 1 → «نجمة واحدة»، 2 → «نجمتان»، 3–10 → «N نجوم»، وما فوق → «N نجمة». لأن العربية لها مفرد ومثنى وجمع.

الأسطر 101–107

```
  /// «متتالية» + «يومًا» 11 / أيام … 10 أيام / 3 / يومان / يوم واحد.
  static String streakDays(int n) {
    if (n == 1) return 'يوم واحد متتالٍ';
    if (n == 2) return 'يومان متتاليان';
    if (n >= 3 && n <= 10) return '$n أيام متتالية';
    return '$n يومًا متتاليًا';
  }
```

ماذا يفعل؟ `streakDays(n)` : صيغة الجمع للأيام المتتالية.

لماذا بهذه الطريقة؟ نفس منطق النجوم لكن بصيغة «يوم/يومان/أيام/يومًا».

الأسطر 109–112

```
  // شاشة الرمز السري (27)
  static const pinEnterTitle = 'أدخل رمزك السري';
  static const pinForgot = 'نسيت الرمز؟ استخدم رمز الطالب لإعادة تعيينه';
  static const pinWrong = 'الرمز غير صحيح، حاول مرة أخرى.';
```

ماذا يفعل؟ نصوص شاشة الرمز السري: العنوان و«نسيت الرمز؟» ورسالة الخطأ.

لماذا بهذه الطريقة؟ من تصميم شاشة 27.

الأسطر 114–116

```
  static String pinHello(String name) => 'مرحبًا $name';
  static String pinLocked(int seconds) =>
      'تجاوزت عدد المحاولات. حاول بعد $seconds ثانية.';
```

ماذا يفعل؟ `pinHello` و `pinLocked` .

لماذا بهذه الطريقة؟ الأولى ترحّب بالاسم، والثانية تذكر الثواني المتبقية للقفل.

الأسطر 118–126

```
  // استعادة الوصول بدل الرمز المنسي
  static const forgotTitle = 'استعادة الوصول';
  static const forgotCodeLabel = 'أدخل رمز الطالب';
  static const forgotCodeHint = 'RZ-XXXX';
  static const forgotCodeWrong = 'رمز الطالب غير صحيح.';
  static const forgotVerify = 'تحقّق';
  static const forgotChoose = 'تم التحقق. ماذا تريد أن تفعل؟';
  static const forgotSetNew = 'تعيين رمز سري جديد';
  static const forgotRemove = 'إلغاء الرمز السري';
```

ماذا يفعل؟ نصوص نافذة «استعادة الوصول» بعد نسيان الرمز.

لماذا بهذه الطريقة؟ تحقّق من رمز الطالب ثم اختيار تعيين رمز جديد أو إلغائه.

الأسطر 128–148

```
  // الرئيسية (03) وأول يوم (28)
  static const homeAskToday = 'ماذا تريد أن تتعلم اليوم؟';
  static const homeFirstDaySubtitle = 'يومك الأول في رزمة. لنبدأ معًا!';
  static const homeStartLesson = 'ابدأ الدرس';
  static const homeContinue = 'متابعة التعلم';
  static const homeFirstSteps = 'خطواتك الأولى';
  static const homeStepDownload = 'نزّل أول درس';
  static const homeStepDownloadHint = 'لتتعلّم حتى دون إنترنت';
  static const homeStepPractice = 'أكمل أول تدريب';
  static const homeStepPracticeHint = 'واربح نجومك الأولى';
  static const homeStepQuiz = 'جرّب أول اختبار';
  static const homeStepQuizHint = 'لتعرف ما تحتاج مراجعته';
  static const homeReviewPlaceholder =
      'ما يحتاج إلى مراجعة سيظهر هنا بعد أول اختبار.';
  static const homeNeedsReview = 'يحتاج إلى مراجعة';
  static const homeReviewButton = 'راجع';
  static const homeMySubjects = 'موادي';
  static const homeShowAll = 'عرض الكل';
  static const homeNotStarted = 'لم تبدأ';
  static const homeLoadFailed = 'تعذّر تحميل الرئيسية.';
  static const homeLessonSoon = 'مشغّل الدروس قيد الإعداد وسيتوفّر قريبًا.';
```

ماذا يفعل؟ نصوص الرئيسية: التحية والخطوات الأولى والمراجعة وموادي وحالات التحميل.

لماذا بهذه الطريقة؟ تشمل رسالة «مشغّل الدروس قيد الإعداد» التي تظهر للأزرار غير المنفَّذة بعد.

الأسطر 150–160

```
  static String homeGreeting(String name) => 'مرحبًا، $name';
  static String homeFirstDayGreeting(String name) => 'أهلًا، $name';
  static String homeStartFirstLesson(String title) => 'ابدأ أول درس: $title';
  static String homeLessonMeta(int minutes) =>
      'نحو $minutes دقيقة · شرح وتدريب قصير';
  static String homeLessonTitle(int order, String title) =>
      'الدرس $order: $title';
  static String homeSubjectUnit(String subject, String unit) =>
      '$subject · $unit';
  static String homeFirstStepsCount(int done) => '$done من 3';
  static String homeMastery(int percent) => 'الإتقان $percent%';
```

ماذا يفعل؟ دوال نصوص الرئيسية المركّبة: التحية وأول درس والمدة وعنوان الدرس و«المادة · الوحدة» وعدّاد الخطوات والإتقان.

لماذا بهذه الطريقة؟ كل دالة تدخل متغيّرًا في الجملة. `homeFirstStepsCount` تقول «X من 3».

الأسطر 162–166

```
  // الشريط السفلي
  static const navHome = 'الرئيسية';
  static const navSubjects = 'المواد';
  static const navProgress = 'تقدّمي';
  static const navProfile = 'حسابي';
```

ماذا يفعل؟ أسماء تبويبات الشريط السفلي.

لماذا بهذه الطريقة؟ الرئيسية · المواد · تقدّمي · حسابي.

السطر 167

```
167    static const navSectionSoon = 'هذا القسم قيد الإعداد وسيتوفّر قريبًا.';
```

ماذا يفعل؟ `navSectionSoon`: رسالة «هذا القسم قيد الإعداد».

لماذا بهذه الطريقة؟ تظهر عند ضغط تبويب «المواد» أو «تقدّمي» ولم يُبنيا بعد؛ أصدق من تجاهل الضغطة بصمت.

الأسطر 169–177

```
169    // مجموعة «الملف» — (19) حسابي
170    static const accountTitle = 'حسابي';
171    static const accountProfileGroup = 'الملف';
172    static const accountStudentCode = 'رمز الطالب';
173    static const accountSwitch = 'تبديل الطالب';
174    static const accountPin = 'الرمز السري';
175    static const accountPinOff = 'غير مفعّل';
176    static const accountPinOn = 'مفعّل';
177    static const accountLoadFailed = 'تعذّر تحميل حسابك.';
```

ماذا يفعل؟ نصوص «حسابي» (19): العنوان واسم مجموعة «الملف» وعناوين الصفوف الثلاثة وحالتا الرمز السري «غير مفعّل/مفعّل» ورسالة فشل التحميل.

لماذا بهذه الطريقة؟ كلها في هذا الملف مثل بقية الشاشات، فلا نص عربي مبعثر داخل الودجتات.

الأسطر 179–180

```
179    static String accountSummary(int grade, int stars) =>
180        '${gradeName(grade)} · ${ArStrings.stars(stars)}';
```

ماذا يفعل؟ `accountSummary(grade, stars)`: «الصف الخامس · 120 نجمة».

لماذا بهذه الطريقة؟ تعيد استعمال `gradeName` و`stars` فتبقى صيغة الجمع العربية صحيحة. لا نكتب «شارات» لأن الخزانة خارج النطاق ولا بيانات لها.

الأسطر 182–188

```
182    // إدارة الرمز السري من حسابي
183    static const pinManageTitle = 'الرمز السري';
184    static const pinChange = 'تغيير الرمز';
185    static const pinDisable = 'إلغاء الرمز السري';
186    static const pinEnabledNote = 'تم تفعيل الرمز السري.';
187    static const pinChangedNote = 'تم تغيير الرمز السري.';
188    static const pinDisabledNote = 'أُلغي الرمز السري.';
```

ماذا يفعل؟ نصوص إدارة الرمز السري من حسابي: العنوان و«تغيير الرمز» و«إلغاء الرمز السري» وثلاث رسائل تأكيد.

لماذا بهذه الطريقة؟ رسالة لكل نتيجة (تفعيل/تغيير/إلغاء) ليعرف الطالب أن ما فعله حدث.

الأسطر 190–204

```
  // (29) رمز الطالب
  static const codeTitle = 'رمز الطالب';
  static const codeYours = 'رمزك';
  static const codeCopy = 'نسخ';
  static const codeShare = 'مشاركة';
  static const codeCopied = 'تم نسخ الرمز.';
  static const codeWhyTitle = 'لماذا هذا الرمز؟';
  static const codeWhy1 = 'انقل تقدّمك إلى جهاز جديد';
  static const codeWhy2 = 'معلمك يضيفك إلى مجموعته';
  static const codeWhy3 = 'لا يحتاج بريدًا ولا رقم هاتف';
  static const codeNeverSynced = 'لم يُحفظ على الإنترنت بعد';
  static const codeNothingPending = 'لا توجد نتائج بانتظار الاتصال';
  static const codeClassGroup = 'مجموعة الصف';
  static const codeKeepSafe =
      'احتفظ بالرمز في مكان آمن، أو اطلب من معلمك تسجيله.';
```

ماذا يفعل؟ نصوص «رمز الطالب» (29): العنوان وأزرار النسخ والمشاركة ورسالة «تم نسخ الرمز» وأسباب وجود الرمز الثلاثة وحالة المزامنة ومجموعة الصف والتذكير بحفظ الرمز.

لماذا بهذه الطريقة؟ «لم يُحفظ على الإنترنت بعد» صادقة: لا مزامنة بعد، فلا نعرض «آخر حفظ: أمس» من التصميم.

الأسطر 206–211

```
  static String codePending(int n) => switch (n) {
    1 => 'نتيجة واحدة بانتظار الاتصال',
    2 => 'نتيجتان بانتظار الاتصال',
    >= 3 && <= 10 => '$n نتائج بانتظار الاتصال',
    _ => '$n نتيجة بانتظار الاتصال',
  };
```

ماذا يفعل؟ `codePending(n)`: «نتيجة واحدة / نتيجتان / 3 نتائج … 10 نتائج / 11 نتيجة» بانتظار الاتصال.

لماذا بهذه الطريقة؟ تعبير `switch` بأنماط النطاق (`>= 3 && <= 10`) يعطي الجمع العربي بوضوح. `_` للباقي. العدد من طابور المزامنة الحقيقي.

الأسطر 212–213

```
  static String codeClassLine(String code) => 'رمز الصف: $code';
  static String codeShareMessage(String code) => 'رمز الطالب في رزمة: $code';
```

ماذا يفعل؟ `codeClassLine` و`codeShareMessage`: سطر رمز الصف ونص رسالة المشاركة.

لماذا بهذه الطريقة؟ دالتان لأن فيهما قيمة متغيّرة (الرمز).

الأسطر 215–229

```
  // أسماء الصفوف
  static const _gradeOrdinals = [
    'الأول',
    'الثاني',
    'الثالث',
    'الرابع',
    'الخامس',
    'السادس',
    'السابع',
    'الثامن',
    'التاسع',
    'العاشر',
    'الحادي عشر',
    'الثاني عشر',
  ];
```

ماذا يفعل؟ قائمة الأعداد الترتيبية العربية للصفوف 1–12.

لماذا بهذه الطريقة؟ قائمة صريحة أسلم من توليدها برمجيًا لأن «الحادي عشر» و«الثاني عشر» لها قواعدها.

الأسطر 231–233

```
  /// مثال: الصف الخامس
  static String gradeName(int grade) => 'الصف ${_gradeOrdinals[grade - 1]}';
}
```

ماذا يفعل؟ `gradeName`: «الصف الخامس» من الرقم 5، ثم إغلاق الصنف.

لماذا بهذه الطريقة؟ `grade - 1` لأن الفهرس يبدأ من الصفر.